

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Факультет безопасности информационных технологий

Дисциплина:

«Технологии и методы программирования»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №5

Выполнили:

Бардышев Артём Антонович,
студент группы N3346

(подпись)

Проверил:

Ищенко Алексей Петрович,
преподаватель, ФБИТ

(отметка о выполнении)

(подпись)

Санкт-Петербург

2025 г.

СОДЕРЖАНИЕ

Введение.....	4
1 Вводные	6
1.1 Шифр Скитала (перестановка).....	6
1.2 Решётка Кардано	8
2 Автоматическое определение режима.....	11
2.1 Функциональность программы	11
3 Технические детали	13
Заключение.....	14
Список использованных источников	Ошибка! Закладка не определена.

ВВЕДЕНИЕ

Цель работы – Реализация исторических алгоритмов шифрования: «Решётка Кардано» и «Шифр перестановки Скитала» с графическим интерфейсом. Изучение принципов работы перестановочных шифров, их криптографических свойств и создание удобного инструмента для демонстрации и практического применения этих алгоритмов.

Исторические шифры перестановки

Перестановочные шифры — это класс криптографических алгоритмов, которые не изменяют символы исходного текста, а только меняют их порядок. В отличие от шифров замены, они сохраняют частоты символов, что делает их уязвимыми для частотного анализа.

Основные свойства:

- Сохранение частот символов
- Обратимость операции
- Простота реализации
- Историческая значимость

Шифр Скитала

Историческая справка:

Скитала (греч. σκυτάλη) — древнегреческий шифр, использовавшийся спартанцами для передачи секретных сообщений. Представляет собой цилиндр с намотанной на него лентой, на которой записывался текст.

Математическая модель:

Текст записывается построчно в матрицу размером $\text{rows} \times \text{cols}$, где $\text{cols} = \text{ceil}(\text{len}(\text{text}) / \text{rows})$, затем считывается по столбцам.

Алгоритм шифрования:

1. Текст разбивается на блоки по rows символов
2. Каждый блок записывается в матрицу построчно
3. Матрица дополняется символами-заполнителями до полного прямоугольника
4. Текст считывается по столбцам сверху вниз

Алгоритм расшифрования:

1. Текст разбивается на блоки по cols символов
2. Каждый блок записывается в матрицу по столбцам
3. Текст считывается построчно
4. Удаляются символы-заполнители

Решётка Кардано

Историческая справка:

Решётка Кардано (итал. griglia di Cardano) — шифр, изобретенный итальянским математиком Джироламо Кардано в XVI веке. Использует физический трафарет с отверстиями для записи текста.

Математическая модель:

Текст записывается в квадратную матрицу $n \times n$ через отверстия в решётке, которая поворачивается 4 раза на 90° по часовой стрелке. Каждая клетка используется ровно один раз.

Условие корректности:

При 4 поворотах решётки все n^2 клеток должны быть покрыты ровно один раз, без пересечений и пропусков.

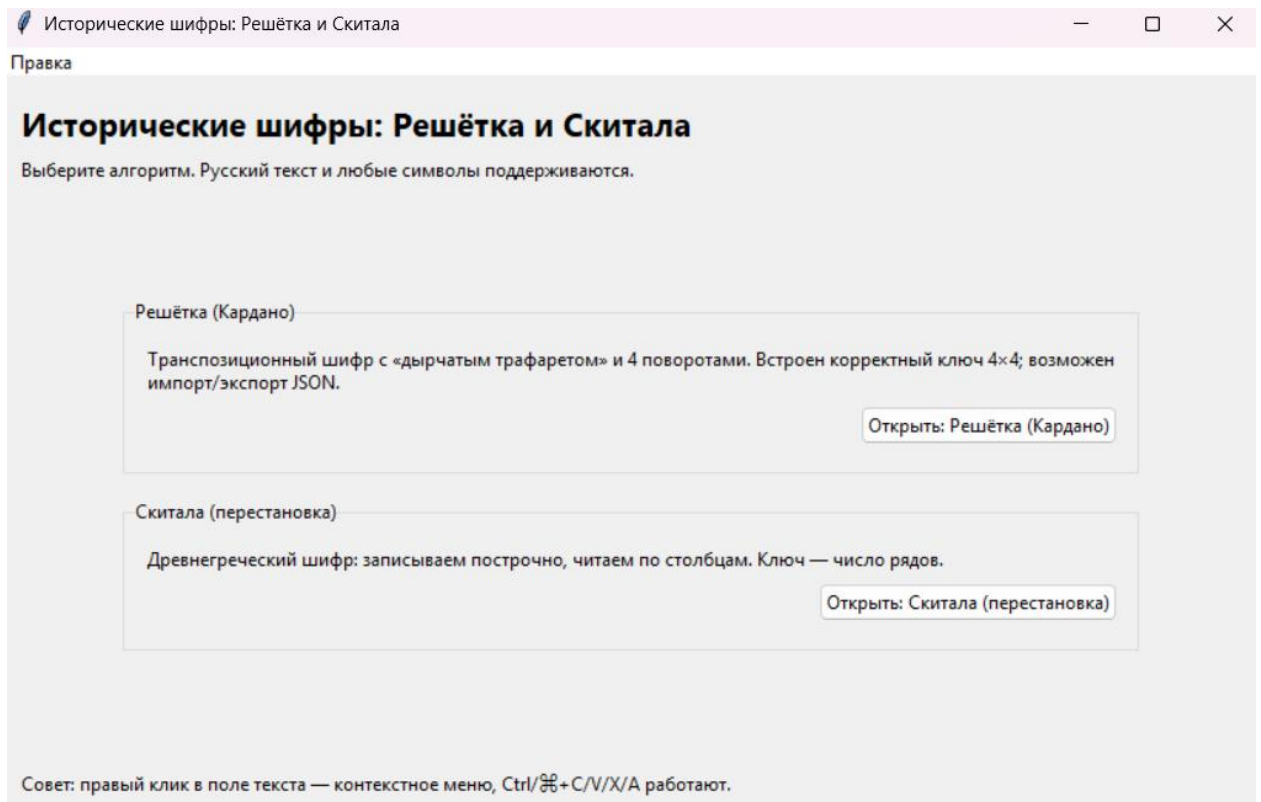
Алгоритм шифрования:

1. Текст разбивается на блоки по n^2 символов
2. Для каждого блока:
 - Создается матрица $n \times n$
 - Решётка поворачивается 4 раза
 - На каждом повороте через отверстия записываются символы
3. Матрица дополняется символами-заполнителями
4. Результат считывается построчно

Алгоритм расшифрования:

1. Текст разбивается на блоки по n^2 символов
2. Для каждого блока:
 - Создается матрица $n \times n$ и заполняется символами шифртекста
 - Решётка поворачивается 4 раза
 - На каждом повороте через отверстия считываются символы

1 ВВОДНЫЕ



1.1 Шифр Скитала (перестановка)

Математическое описание

Для текста длиной n и количеством рядов r :

- Количество столбцов: $c = \lceil n/r \rceil$
- Размер матрицы: $r \times c$
- Количество символов-заполнителей: $r \times c - n$

Реализация функций

Функция шифрования:

```
def skytale_encrypt(text: str, rows: int, filler: str = " ") -> str:
    if rows <= 1:
        return text
    n = len(text)
    cols = (n + rows - 1) // rows    Округление вверх
    padded = text + (filler * (rows * cols - n))
    grid = [list(padded[icols:(i+1)*cols]) for i in range(rows)]
    out = []
    for c in range(cols):
        for r in range(rows):
            out.append(grid[r][c])
    return "".join(out)
```

Функция расшифрования:

```
def skytale_decrypt(cipher: str, rows: int, filler: str = " ") -> str:
    if rows <= 1:
        return cipher
    n = len(cipher)
    cols = (n + rows - 1) // rows
    grid = [[""] * cols for _ in range(rows)]
    it = iter(cipher)
    for c in range(cols):
        for r in range(rows):
            grid[r][c] = next(it, filler)
    out = "".join("".join(row) for row in grid)
    return out.rstrip(filler)
```

Пример работы

Исходный текст: "ПРИВЕТМИР"

Количество рядов: 3

Шифрование:

Матрица записи:

П Р И

В Е Т

М И Р

Матрица чтения (по столбцам):

П В М

Р Е И

И Т Р

Результат: "ПВМРЕИИТР"

Расшифрование:

Матрица записи (по столбцам):

П В М

Р Е И

И Т Р

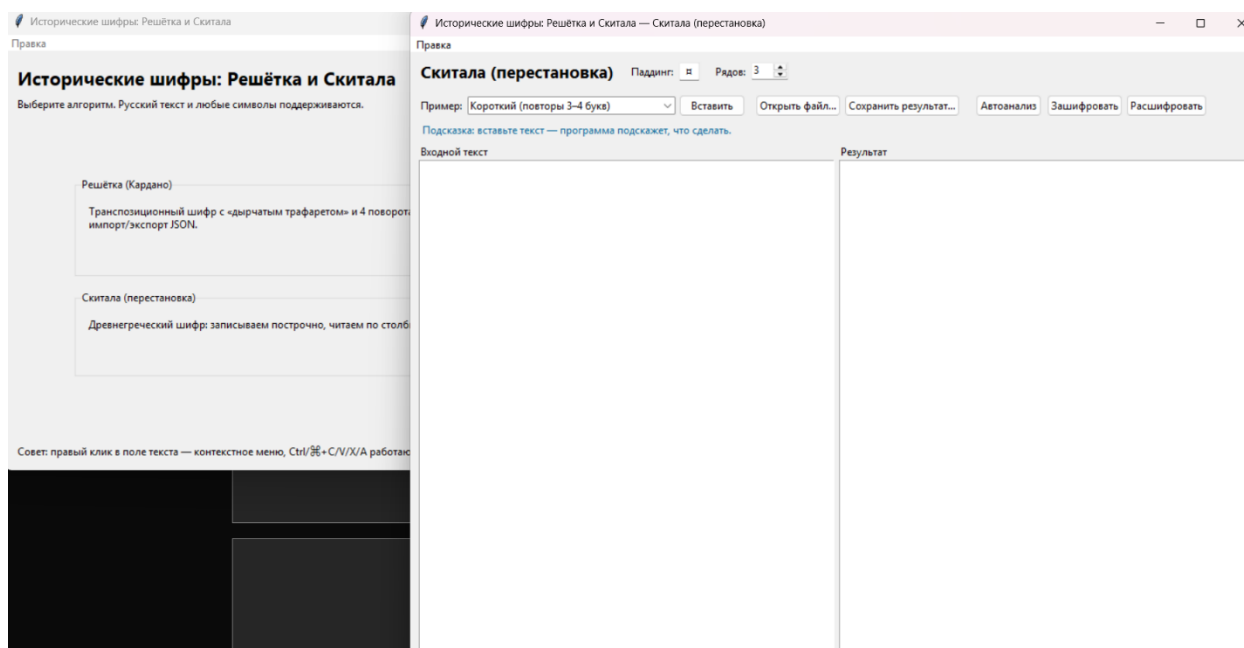
Матрица чтения (построчно):

П Р И

В Е Т

М И Р

Результат: "ПРИВЕТМИР"



1.2 Решётка Кардано

Математическое описание

Для решётки размером $n \times n$:

- Количество отверстий: $n^2/4$ (для покрытия всех клеток за 4 поворота)
- Каждое отверстие используется 4 раза (при 4 поворотах)
- Общее количество использований: $4 \times (n^2/4) = n^2$ ✓

Валидация решётки

Функция проверки корректности:

```
def validate_grille(coords, n):
    """Проверяет, что при 4 поворотах покрываются все nn клетки ровно один
    раз."""
    used = set()
    cur = list(coords)
    for _ in range(4):
        for cell in cur:
            if cell in used:
                return False    Пересечение
            used.add(cell)
        cur = rotate_coords(cur, n)    Поворот на 90°
    return len(used) == n    n    Все клетки покрыты
```

Функция поворота координат:

```
def rotate_coords(coords, n):
    """Поворот координат на 90° по часовой стрелке."""
    return [(c, n-1-r) for (r, c) in coords]
```

Встроенная решётка 4×4

Координаты отверстий:

```
DEFAULT_GRILLE = {
    "n": 4,
    "holes": [(0,0), (0,1), (0,2), (1,1)]
```

}

Визуализация решётки:

Поворот 0°: Поворот 90°: Поворот 180°: Поворот 270°:

```
[•][•][•][ ] [ ][•][ ][ ] [ ][•][ ][ ] [•][ ][ ][ ]
[ ][•][ ][ ] [•][•][ ][ ] [ ][•][ ][ ] [ ][•][ ][ ]
[ ][ ][ ][ ] [ ][ ][ ][ ] [ ][ ][ ][ ] [ ][ ][ ][ ]
[ ][ ][ ][ ] [ ][ ][ ][ ] [ ][ ][ ][ ] [ ][ ][ ][ ]
```

Реализация функций

Функция шифрования:

```
def grille_encrypt(text: str, grille: dict, filler: str = "x") -> str:
    n = grille["n"]
    holes = grille["holes"]
    if not validate_grille(holes, n):
        raise ValueError("Ключ-решётка некорректна.")
    out = []
    i = 0
    while i < len(text):
        block = text[i:i+nn]
        if len(block) < nn:
            block = block + filler * (nn - len(block))
        grid = ["" * n for _ in range(n)]
        idx = 0
        cur = list(holes)
        for _ in range(4):  # 4 поворота
            for (r, c) in cur:
                grid[r][c] = block[idx]
                idx += 1
            cur = rotate_coords(cur, n)
        for r in range(n):
            for c in range(n):
                out.append(grid[r][c])
        i += nn
    return "".join(out)
```

Функция расшифрования:

```
def grille_decrypt(cipher: str, grille: dict, filler: str = "x") -> str:
    n = grille["n"]
    holes = grille["holes"]
    if not validate_grille(holes, n):
        raise ValueError("Ключ-решётка некорректна.")
    out = []
    i = 0
    while i < len(cipher):
        block = cipher[i:i+nn]
        if len(block) < nn:
            block = block + filler * (nn - len(block))
        grid = ["" * n for _ in range(n)]
        it = iter(block)
        for r in range(n):
            for c in range(n):
                grid[r][c] = next(it)
        cur = list(holes)
        chars = []
        for _ in range(4):  # 4 поворота
            for (r, c) in cur:
```



```

        chars.append(grid[r][c])
        cur = rotate_coords(cur, n)
        out.append("".join(chars))
        i += nn
    return "".join(out).rstrip(filler)

```

Пример работы

Исходный текст: "ПРИВЕТМИР123456"

Решётка: 4×4 с отверстиями [(0,0), (0,1), (0,2), (1,1)]

Шифрование:

Блок 1: "ПРИВЕТМИР123456" (16 символов)

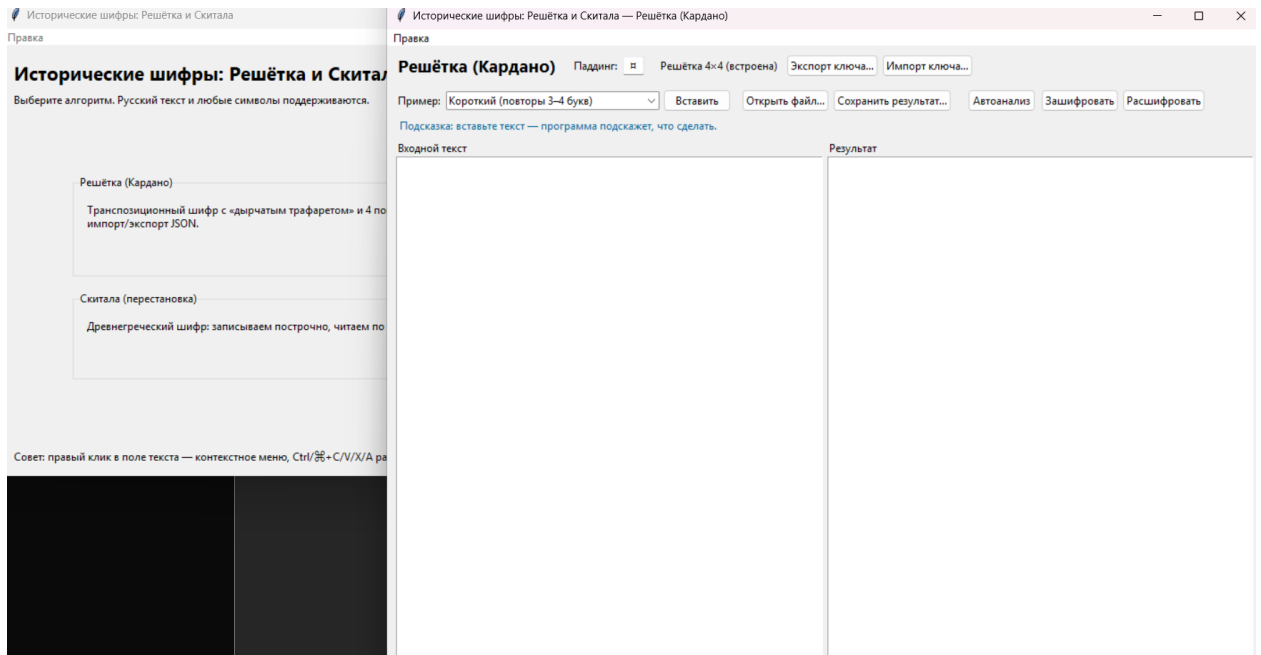
Поворот 0°: запись П, Р, И, В

Поворот 90°: запись Е, Т, М, И

Поворот 180°: запись Р, 1, 2, 3

Поворот 270°: запись 4, 5, 6, ☐

Результат: "ПЕР4РИТ15ИМ236ВИ☐☐☐☐"



2 АВТОМАТИЧЕСКОЕ ОПРЕДЕЛЕНИЕ РЕЖИМА

Эвристический анализ текста

Программа использует эвристику для определения, является ли текст зашифрованным или открытым.

Функция оценки "натуральности" текста:

```
def russian_natural_score(text: str) -> float:
    """Оценка того, насколько текст похож на нормальный русский текст."""
    Анализ:
    - Доля кириллических символов
    - Доля гласных букв
    - Наличие пробелов
    - Наличие знаков препинания
    - Наличие частых русских слов
    Возвращает значение от 0.0 до 1.0
```

Алгоритм автоопределения:

1. Вычисляется "натуральность" исходного текста
2. Выполняется попытка расшифрования
3. Вычисляется "натуральность" расшифрованного текста
4. Если расшифрованный текст "более натуральный" — текст зашифрован
5. Иначе — текст открытый

2.1 Функциональность программы

Основные возможности

1. Два режима работы
 - Решётка (Кардано) — перестановочный шифр с 4 поворотами квадратной решётки
 - Скитала (перестановка) — древнегреческий шифр с фиксированным количеством рядов
2. Автоматическое определение
 - Программа анализирует вставленный текст
 - Предлагает «Расшифровать» или «Зашифровать повторно»
 - Использует эвристику для определения «натуральности» русского текста
 - Показывает уровень уверенности в определении
3. Работа с файлами
 - Открытие файлов: поддержка текстовых файлов (.txt)
 - Сохранение результатов: экспорт зашифрованного/расшифрованного текста
 - Импорт/экспорт ключей решётки: сохранение и загрузка ключей в формате JSON

4. Поддержка любого текста

- Русский, английский, смешанный текст — полная поддержка Unicode
- Цифры, символы, эмодзи — сохранение всех символов
- Знаки препинания и регистры — полное сохранение форматирования
- Любая длина текста — автоматическое разбиение на блоки

5. Удобный интерфейс

Меню "Правка":

- Вырезать (Ctrl+X)
- Копировать (Ctrl+C)
- Вставить (Ctrl+V)
- Выделить всё (Ctrl+A)
- Очистить

Контекстное меню (ПКМ):

- Те же операции доступны через правый клик
- Работает в любом текстовом поле

Горячие клавиши:

- Работают в любой раскладке (русской/английской)
- Стандартные комбинации: Ctrl+C/V/X/A
- Альтернативные: Ctrl/Shift+Insert, Shift+Delete

Дополнительные возможности:

- Автоматический перенос строк
- Масштабируемое окно
- Undo/Redo для текстовых полей
- Подсказки и статусная строка

6. Настройки

- Изменение символа-заполнителя (падинг) — по умолчанию "□", можно изменить
- Настройка количества рядов для Считалы — от 2 до 50
- Импорт/экспорт ключей решётки — сохранение и загрузка в JSON

3 ТЕХНИЧЕСКИЕ ДЕТАЛИ

Используемые технологии

- Язык: Python 3.9+
- GUI: `tkinter` (встроен в Python, кроссплатформенный)
- Сборка exe: `PyInstaller`
- Дополнительные библиотеки: не требуются (только стандартная библиотека Python)

Структура проекта

5/

```
|— main.py          Основной файл программы
|— README.md        Документация
|— Шифровальщик.exe Собранный исполняемый файл
└— (опционально) app.ico Иконка приложения
```

Сборка исполняемого файла

Установка PyInstaller:

```
python -m pip install pyinstaller
```

Команда сборки:

```
pyinstaller --noconsole --onefile --name "Шифровальщик" main.py
```

Параметры:

- `--noconsole` — скрыть консольное окно (только GUI)
- `--onefile` — собрать в один исполняемый файл
- `--name "Шифровальщик"` — имя выходного файла

Результат:

- Файл `Шифровальщик.exe` в папке `dist/`
- Размер: ~8-12 МБ (включая Python runtime)

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы были изучены и реализованы:

Теоретические знания

1. Принципы работы исторических шифров:

- Перестановочные шифры и их свойства
- Математические модели алгоритмов
- Криптографические свойства

2. Алгоритм Скитала:

- Исторический контекст
- Математическая модель
- Реализация и оптимизация

3. Алгоритм решётки Кардано:

- Принцип работы с трафаретом
- Валидация ключа
- Реализация поворотов

4. Создание пользовательского интерфейса:

- Работа с tkinter
- Обработка событий
- Удобство использования

5. Методы автоматического определения:

- Эвристический анализ
- Оценка "натуральности" текста
- Принятие решений

Практические навыки

1. Программирование:

- Реализация криптографических алгоритмов
- Работа с матрицами и координатами
- Обработка текстовых данных

2. Разработка GUI:

- Создание интерфейсов на tkinter
- Обработка пользовательского ввода
- Визуальная обратная связь

3. Упаковка приложений:

- Использование PyInstaller
- Создание автономных исполняемых файлов
- Оптимизация размера

Криптографический анализ

Сильные стороны алгоритмов:

- Простота реализации
- Быстрота работы
- Обратимость операций
- Историческая значимость

Слабые стороны:

- Сохранение частот символов (уязвимость к частотному анализу)
- Относительно простая структура
- Не подходят для современных задач безопасности

Применение:

- Образовательные цели
- Демонстрация принципов криптографии
- Историческое изучение
- Не для реальной защиты данных